

Django Vue Lab

Лекция 5. Django REST Framework

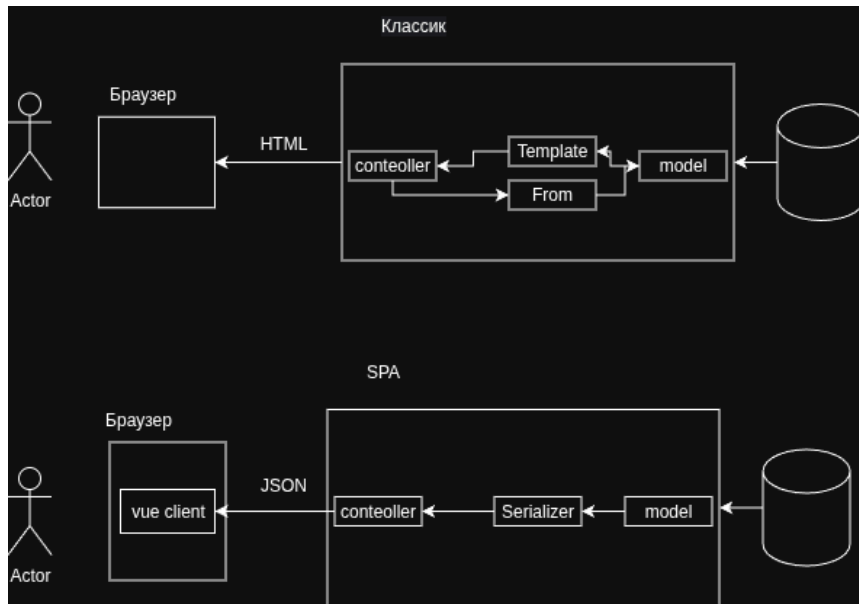
Черепанов И.Н. Филиппев В.А.

Содержание

REST	3
Сериализатор	11
Controllers	23
Права доступа	38

REST

Зачем нужен API



Django REST Framework

Django REST Framework, DRF – библиотека для создания HTTP API на Django.

- превращает модели Django в JSON-представление
- проверяет входные данные
- связывает HTTP-методы с CRUD-операциями
- даёт готовые классы view и роутеры
- поддерживает аутентификацию, права доступа, пагинацию
- предоставляет browsable API для разработки

DRF не заменяет Django, а добавляет слой API поверх обычного Django-проекта.

HTML view и API view

Обычный Django view часто возвращает HTML.

```
from django.shortcuts import render

def article_list(request):
    articles = Article.objects.all()
    return render(request, "articles/list.html", {"articles": articles})
```

API view возвращает данные, обычно JSON.

```
from rest_framework.response import Response

def article_list(request):
    articles = Article.objects.all()
    data = [{"id": item.id, "title": item.title} for item in articles]
    return Response(data)
```

REST

REST – стиль проектирования API, где ресурсы доступны по URL, а действия выражаются HTTP-методами.

GET	/api/articles/	список статей
POST	/api/articles/	создать статью
GET	/api/articles/42/	получить одну статью
PUT	/api/articles/42/	заменить статью
PATCH	/api/articles/42/	частично изменить статью
DELETE	/api/articles/42/	удалить статью

URL описывает ресурс, метод описывает действие.

HTTP-статусы

API должен явно сообщать результат запроса через статус.

- 200 OK – успешный запрос
- 201 Created – объект создан
- 204 No Content – успешно, тело ответа не нужно
- 400 Bad Request – ошибка входных данных
- 401 Unauthorized – пользователь не аутентифицирован
- 403 Forbidden – нет прав
- 404 Not Found – объект не найден
- 500 Internal Server Error – ошибка сервера

Установка

```
pip install djangorestframework
```

```
# settings.py
INSTALLED_APPS = [
    # ...
    "rest_framework",
]
```

После этого можно использовать сериализаторы, API views, роутеры и browsable API.

Модель для примеров

```
from django.db import models
from django.contrib.auth import get_user_model

User = get_user_model()

class Article(models.Model):
    title = models.CharField(max_length=200)
    text = models.TextField()
    author = models.ForeignKey(User, on_delete=models.CASCADE)
    is_published = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

Эта модель будет ресурсом `/api/articles/`.

Сериализатор

Сериализатор

Сериализатор решает две задачи.

- Python object -> JSON-compatible dict
- входной JSON -> проверенные Python-значения

```
from rest_framework import serializers

class ArticleSerializer(serializers.Serializer):
    id = serializers.IntegerField(read_only=True)
    title = serializers.CharField(max_length=200)
    text = serializers.CharField()
    is_published = serializers.BooleanField(default=False)
```

Сериализатор похож на Django Form, но работает с данными API.

Serializer: ВЫВОД ДАННЫХ

```
article = Article.objects.get(pk=1)
serializer = ArticleSerializer(article)

serializer.data
```

```
{
    "id": 1,
    "title": "Django REST Framework",
    "text": "API на Django",
    "is_published": True,
}
```

`serializer.data` содержит данные, которые можно отдать клиенту как JSON.

Serializer: входные данные

```
data = {  
    "title": "Новая статья",  
    "text": "Текст статьи",  
    "is_published": False,  
}  
  
serializer = ArticleSerializer(data=data)  
  
if serializer.is_valid():  
    print(serializer.validated_data)  
else:  
    print(serializer.errors)
```

validated_data – безопасные данные после проверки типов и ограничений.

Валидация поля

```
from rest_framework import serializers

class ArticleSerializer(serializers.Serializer):
    title = serializers.CharField(max_length=200)
    text = serializers.CharField()

    def validate_title(self, value):
        if "spam" in value.lower():
            raise serializers.ValidationError("Недопустимое слово")
        return value
```

Метод `validate_<field>()` проверяет одно поле.

Валидация объекта

```
class ArticleSerializer(serializers.Serializer):
    title = serializers.CharField(max_length=200)
    text = serializers.CharField()
    is_published = serializers.BooleanField(default=False)

    def validate(self, attrs):
        if attrs["is_published"] and len(attrs["text"]) < 100:
            raise serializers.ValidationError(
                "Короткую статью нельзя публиковать"
            )
        return attrs
```

Метод `validate()` проверяет связь между несколькими полями.

ModelSerializer

ModelSerializer строит поля по модели Django.

```
from rest_framework import serializers
from .models import Article

class ArticleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Article
        fields = [
            "id",
            "title",
            "text",
            "author",
            "is_published",
            "created_at",
        ]
        read_only_fields = ["id", "created_at"]
```

Это самый частый вариант сериализатора в DRF.

fields и exclude

Лучше явно перечислять поля, которые доступны через API.

```
class ArticleSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Article  
        fields = ["id", "title", "text", "is_published", "created_at"]
```

```
class ArticleSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Article  
        exclude = ["author"]
```

`fields = "__all__"` удобно для черновика, но рискованно для публичного API.

read_only и write_only

```
class UserSerializer(serializers.ModelSerializer):  
    password = serializers.CharField(write_only=True)  
  
    class Meta:  
        model = User  
        fields = ["id", "username", "email", "password"]  
        read_only_fields = ["id"]
```

- read_only – поле есть в ответе, но не принимается от клиента
- write_only – поле принимается от клиента, но не отдаётся в ответе

Так защищают технические и чувствительные поля.

Связи между моделями

По умолчанию ForeignKey часто отображается как id связанного объекта.

```
class ArticleSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Article  
        fields = ["id", "title", "author"]
```

```
{  
    "id": 1,  
    "title": "DRF",  
    "author": 7  
}
```

Это удобно для записи и простого API.

Вложенный сериализатор

```
class AuthorSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = User  
        fields = ["id", "username"]  
  
class ArticleSerializer(serializers.ModelSerializer):  
    author = AuthorSerializer(read_only=True)  
  
    class Meta:  
        model = Article  
        fields = ["id", "title", "author"]
```

```
{  
    "id": 1,  
    "title": "DRF",  
    "author": {"id": 7, "username": "anna"}  
}
```

create и update

Обычный Serializer должен явно знать, как создать и обновить объект.

```
class ArticleSerializer(serializers.Serializer):
    title = serializers.CharField(max_length=200)
    text = serializers.CharField()

    def create(self, validated_data):
        return Article.objects.create(**validated_data)

    def update(self, instance, validated_data):
        instance.title = validated_data.get("title", instance.title)
        instance.text = validated_data.get("text", instance.text)
        instance.save()
        return instance
```

ModelSerializer генерирует базовые create() и update() автоматически.

Controllers

Function-based API view

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Article
from .serializers import ArticleSerializer

@api_view(["GET"])
def article_list(request):
    articles = Article.objects.all()
    serializer = ArticleSerializer(articles, many=True)
    return Response(serializer.data)
```

@api_view превращает обычную функцию в DRF view.

POST в function-based view

```
from rest_framework import status

@api_view(["GET", "POST"])
def article_list(request):
    if request.method == "GET":
        articles = Article.objects.all()
        serializer = ArticleSerializer(articles, many=True)
        return Response(serializer.data)

    serializer = ArticleSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save(author=request.user)
        return Response(serializer.data, status=status.HTTP_201_CREATED)

    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

APIView

APIView похож на class-based view Django, но работает с DRF Request и Response.

```
from rest_framework.views import APIView
from rest_framework.response import Response

class ArticleListAPIView(APIView):
    def get(self, request):
        articles = Article.objects.all()
        serializer = ArticleSerializer(articles, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = ArticleSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        serializer.save(author=request.user)
        return Response(serializer.data, status=201)
```

Detail APIView

```
from django.shortcuts import get_object_or_404

class ArticleDetailAPIView(APIView):
    def get(self, request, pk):
        article = get_object_or_404(Article, pk=pk)
        serializer = ArticleSerializer(article)
        return Response(serializer.data)

    def delete(self, request, pk):
        article = get_object_or_404(Article, pk=pk)
        article.delete()
        return Response(status=204)
```

В таком стиле вся логика HTTP-методов пишется вручную.

Generic views

Generic views убирают повторяющийся код.

```
from rest_framework import generics
from .models import Article
from .serializers import ArticleSerializer

class ArticleListCreateView(generics.ListCreateAPIView):
    queryset = Article.objects.all()
    serializer_class = ArticleSerializer

class ArticleDetailView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Article.objects.all()
    serializer_class = ArticleSerializer
```

DRF сам реализует list, create, retrieve, update, destroy.

get_queryset

`get_queryset()` используют, когда набор объектов зависит от запроса.

```
class MyArticlesView(generics.ListAPIView):  
    serializer_class = ArticleSerializer  
  
    def get_queryset(self):  
        return Article.objects.filter(author=self.request.user)
```

```
class PublishedArticlesView(generics.ListAPIView):  
    serializer_class = ArticleSerializer  
  
    def get_queryset(self):  
        return Article.objects.filter(is_published=True)
```

perform_create

`perform_create()` нужен, когда часть данных берётся не из JSON, а из запроса.

```
class ArticleListCreateView(generics.ListCreateAPIView):  
    queryset = Article.objects.all()  
    serializer_class = ArticleSerializer  
  
    def perform_create(self, serializer):  
        serializer.save(author=self.request.user)
```

Клиент не должен передавать `author`, если автор – текущий пользователь.

ViewSet

ViewSet объединяет действия над ресурсом в одном классе.

```
from rest_framework import viewsets

class ArticleViewSet(viewsets.ModelViewSet):
    queryset = Article.objects.all()
    serializer_class = ArticleSerializer
```

ModelViewSet даёт полный CRUD:

- list
- create
- retrieve
- update
- partial_update
- destroy

URL для generic views

```
from django.urls import path
from . import views

urlpatterns = [
    path("api/articles/", views.ArticleListCreateView.as_view()),
    path("api/articles/<int:pk>/", views.ArticleDetailView.as_view()),
]
```

Такой вариант хорошо подходит, когда URL нужно контролировать вручную.

Router для ViewSet

```
from django.urls import include, path
from rest_framework.routers import DefaultRouter
from .views import ArticleViewSet

router = DefaultRouter()
router.register("articles", ArticleViewSet)

urlpatterns = [
    path("api/", include(router.urls)),
]
```

Роутер сам создаст маршруты для списка и детальной страницы.

Router: результат

```
GET    /api/articles/  
POST   /api/articles/  
GET     /api/articles/{id}/  
PUT     /api/articles/{id}/  
PATCH  /api/articles/{id}/  
DELETE  /api/articles/{id}/
```

DefaultRouter также добавляет корневую страницу API со списком ресурсов.

Request и Response

DRF использует свои классы поверх Django request/response.

```
request.data
```

- данные из JSON
- данные из HTML-формы
- данные из multipart/form-data

```
return Response(data, status=201)
```

Response сам выбирает формат ответа через renderer, обычно JSON.

raise_exception=True

```
serializer = ArticleSerializer(data=request.data)
serializer.is_valid(raise_exception=True)
serializer.save()
```

Если данные невалидны, DRF сам вернёт 400 Bad Request.

```
{
  "title": ["Это поле обязательно."]
}
```

Так код view становится короче и понятнее.

Browsable API

DRF умеет показывать API как HTML-страницы в браузере.

- удобно проверять список объектов
- можно отправлять POST, PUT, PATCH, DELETE
- видно поля сериализатора
- видно ошибки валидации
- удобно для разработки и обучения

Для production browsable API часто ограничивают настройками и правами доступа.

Права доступа

Права доступа

Permission classes решают, кто может выполнить запрос.

```
from rest_framework.permissions import IsAuthenticated

class ArticleViewSet(viewsets.ModelViewSet):
    queryset = Article.objects.all()
    serializer_class = ArticleSerializer
    permission_classes = [IsAuthenticated]
```

Теперь API доступен только аутентифицированным пользователям.

Встроенные permissions

- AllowAny – доступ всем
- IsAuthenticated – только вошедшим пользователям
- IsAdminUser – только администраторам
- IsAuthenticatedOrReadOnly – чтение всем, изменение только вошедшим
- DjangoModelPermissions – права Django на модель

```
from rest_framework.permissions import IsAuthenticatedOrReadOnly  
  
permission_classes = [IsAuthenticatedOrReadOnly]
```

Собственный permission

```
from rest_framework.permissions import BasePermission, SAFE_METHODS

class IsAuthorOrReadOnly(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        return obj.author == request.user
```

```
class ArticleViewSet(viewsets.ModelViewSet):
    permission_classes = [IsAuthorOrReadOnly]
```

Так можно разрешить редактирование только автору объекта.

Аутентификация

DRF поддерживает разные способы аутентификации.

- `SessionAuthentication` – обычная Django-сессия и cookie
- `BasicAuthentication` – логин и пароль в HTTP-заголовке
- `TokenAuthentication` – токен в HTTP-заголовке
- JWT – часто через сторонние пакеты

Для Vue и SPA часто используют cookie-сессию с CSRF или token/JWT в зависимости от проекта.

Настройки DRF

```
# settings.py
REST_FRAMEWORK = {
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticatedOrReadOnly",
    ],
    "DEFAULT_PAGINATION_CLASS":
        "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 20,
}
```

Глобальные настройки можно переопределять в конкретном view.

Фильтрация вручную

```
class ArticleListView(generics.ListAPIView):
    serializer_class = ArticleSerializer

    def get_queryset(self):
        queryset = Article.objects.all()
        published = self.request.query_params.get("published")

        if published == "true":
            queryset = queryset.filter(is_published=True)

        return queryset
```

GET /api/articles/?published=true

django-filter

```
pip install django-filter
```

```
INSTALLED_APPS = [  
    # ...  
    "django_filters",  
]
```

```
from django_filters.rest_framework import DjangoFilterBackend  
  
class ArticleViewSet(viewsets.ModelViewSet):  
    queryset = Article.objects.all()  
    serializer_class = ArticleSerializer  
    filter_backends = [DjangoFilterBackend]  
    filterset_fields = ["is_published", "author"]
```

Поиск

```
from rest_framework import filters

class ArticleViewSet(viewsets.ModelViewSet):
    queryset = Article.objects.all()
    serializer_class = ArticleSerializer
    filter_backends = [filters.SearchFilter]
    search_fields = ["title", "text"]
```

```
GET /api/articles/?search=django
```

SearchFilter делает простой поиск по указанным полям.

Сортировка

```
from rest_framework import filters

class ArticleViewSet(viewsets.ModelViewSet):
    queryset = Article.objects.all()
    serializer_class = ArticleSerializer
    filter_backends = [filters.OrderingFilter]
    ordering_fields = ["created_at", "title"]
    ordering = ["-created_at"]
```

GET /api/articles/?ordering=title

GET /api/articles/?ordering=-created_at

Несколько filter_backends

```
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework import filters

class ArticleViewSet(viewsets.ModelViewSet):
    queryset = Article.objects.all()
    serializer_class = ArticleSerializer
    filter_backends = [
        DjangoFilterBackend,
        filters.SearchFilter,
        filters.OrderingFilter,
    ]
    filterset_fields = ["is_published"]
    search_fields = ["title", "text"]
    ordering_fields = ["created_at", "title"]
```

Пагинация

Пагинация ограничивает количество объектов в одном ответе.

```
REST_FRAMEWORK = {  
    "DEFAULT_PAGINATION_CLASS":  
        "rest_framework.pagination.PageNumberPagination",  
    "PAGE_SIZE": 10,  
}
```

```
GET /api/articles/?page=2
```

Без пагинации большой список может перегрузить сервер и браузер.

Ответ с пагинацией

```
{
  "count": 42,
  "next": "http://localhost:8000/api/articles/?page=3",
  "previous": "http://localhost:8000/api/articles/?page=1",
  "results": [
    {"id": 11, "title": "DRF"}
  ]
}
```

Во Vue нужно читать не только results, но и ссылки next, previous или номер страницы.

API и CSRF

Если Vue работает через Django-сессию, небезопасные методы требуют CSRF-токен.

```
await fetch("/api/articles/", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "X-CSRFToken": getCookie("csrftoken"),
  },
  body: JSON.stringify(form),
});
```

GET, HEAD, OPTIONS обычно не требуют CSRF.

CORS

CORS нужен, если frontend и backend находятся на разных origin.

```
http://localhost:5173 -> Vue dev server  
http://localhost:8000 -> Django API
```

Решения:

- настроить проху в Vite
- использовать django-cors-headers
- в production отдавать frontend и API с одного домена

Для разработки с Vue проще начать с проху.

Vue: загрузка списка

```
<script setup>
import { onMounted, ref } from "vue";

const articles = ref([]);
const isLoading = ref(false);

async function loadArticles() {
  isLoading.value = true;
  const response = await fetch("/api/articles/");
  const data = await response.json();
  articles.value = data.results ?? data;
  isLoading.value = false;
}

onMounted(loadArticles);
</script>
```

Vue: отправка формы

```
async function createArticle(form) {  
  const response = await fetch("/api/articles/", {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify(form),  
  });  
  
  if (!response.ok) {  
    throw await response.json();  
  }  
  
  return await response.json();  
}
```

Ошибки валидации DRF удобно показывать рядом с полями формы.

Ошибки валидации

DRF возвращает ошибки как объект, где ключ – имя поля.

```
{  
  "title": ["Это поле обязательно."],  
  "text": ["Убедитесь, что это поле содержит не менее 100 символов."]  
}
```

Во Vue такие ошибки можно сохранить в состояние формы.

```
errors.value = await response.json();
```

Документация API

DRF даёт browsable API, но для команды и внешних клиентов нужна спецификация.

- OpenAPI описывает endpoints, параметры, тела запросов и ответы
- Swagger UI показывает документацию в браузере
- клиентский код можно генерировать из схемы

Для DRF часто используют `drf-spectacular`.

Минимальный CRUD

```
# serializers.py
class ArticleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Article
        fields = ["id", "title", "text", "is_published", "created_at"]
        read_only_fields = ["id", "created_at"]
```

```
# views.py
class ArticleViewSet(viewsets.ModelViewSet):
    queryset = Article.objects.all()
    serializer_class = ArticleSerializer
    permission_classes = [IsAuthenticatedOrReadOnly]
```

Минимальный CRUD: URL

```
# urls.py
from django.urls import include, path
from rest_framework.routers import DefaultRouter
from .views import ArticleViewSet

router = DefaultRouter()
router.register("articles", ArticleViewSet)

urlpatterns = [
    path("api/", include(router.urls)),
]
```

Этого достаточно, чтобы получить базовый API для модели Article.

Спасибо за внимание

Навык “Web разработка” повышается